

LabReSiD26 – Hands-On 13

Davide Ferrara – 518629

1 Esercizio 1: Somma parallela di un array

L'esercizio consiste nel dividere un array di $N = 1\,000\,000$ interi in $T = 4$ segmenti uguali, calcolare la somma parziale di ciascun segmento in un thread separato tramite `pthread_create()`, raccogliere i risultati con `pthread_join()` e verificare che la somma totale coincida con quella calcolata sequenzialmente.

Struttura degli argomenti

Ogni thread riceve un puntatore a una struttura `worker_args` che descrive il segmento di lavoro e dove scrivere il risultato parziale. Usare una struttura è necessario perché `pthread_create()` accetta un unico argomento `void *`.

```
1  /* Struttura dati del worker*/
2  typedef struct {
3      const int *array;
4      int from;
5      int to;
6      ssize_t result;
7  } worker_args;
8
9  /* Job del thread, fa il casting dell'args da void* in
10     worker_args*
11     * in questo modo il programma sa come leggere la
12       struttura in memoria.
13     * Uso la funziona gettid() per ottenere il thread id.
14     * Il thread chiama partial_sum() e salva il risultato
15       nella struttura,
16     * il thread ritorna NULL.
17     * */
18 static void job(void *args) {
19     worker_args *w = (worker_args *)args;
```

```

17  pid_t tid = gettid();
18
19  w->result = partial_sum(w->array, w->from, w->to);
20  printf("[SUM.c][Thread %d] Somma parziale: %lu\n", tid
    , w->result);
21
22  pthread_exit(NULL);
23 }

```

Listing 1: Struttura argomento thread e funzione job

Creazione e raccolta dei thread

I loop di creazione e raccolta sono separati: il primo lancia tutti i thread in parallelo, il secondo raccoglie i risultati con `pthread_join()`. I segmenti dell'array sono disgiunti, quindi i thread non condividono mai dati in scrittura: non serve nessun meccanismo di sincronizzazione. `pthread_join()` attende la terminazione del thread e raccoglie il valore restituito da `return` o `pthread_exit()`.

```

1  worker_args w[T] = {0};
2  pthread_t tid[T];
3
4  /* Stabilisco lo step per dividere l'array in base a T
   */
5  int step = SIZE / T;
6  int from = 0, to = step;
7
8  /* Lancio tutti i thread in parallelo: segmenti
   disgiunti,
9  * nessuna sincronizzazione necessaria. */
10 for (int i = 0; i < T; i++) {
11     w[i].array = a;
12     w[i].from = from;
13     w[i].to = to;
14     from = to;
15     to += step;
16     pthread_create(&tid[i], NULL, (void *)&job, (void *)&w
        [i]);
17 }
18
19 long long psum = 0;
20 for (int i = 0; i < T; i++) {

```

```

21  if (pthread_join(tid[i], NULL) == 0)
22      psum += w[i].result;
23  }

```

Listing 2: Loop di creazione e raccolta separati

```

1  $ ./sum
2  [SUM.c] Somma sequenziale: 49500000
3  [SUM.c][Thread 29728] Somma parziale: 12375000
4  [SUM.c][Thread 29729] Somma parziale: 12375000
5  [SUM.c][Thread 29730] Somma parziale: 12375000
6  [SUM.c][Thread 29731] Somma parziale: 12375000
7  [SUM.c] Somma parallela (T=4): 49500000
8  [OK] i risultati coincidono!

```

Listing 3: Test Esercizio 1

2 Esercizio 2: Barrier – map-reduce parallelo

L'esercizio implementa uno schema map-reduce in due fasi usando `pthread_barrier_t`: T thread eseguono la fase di *map* in parallelo (calcolo del quadrato di ogni elemento nel proprio segmento), si sincronizzano alla barrier, poi il solo thread che riceve `PTHREAD_BARRIER_SERIAL_THREAD` esegue la fase di *reduce* (somma di tutti i quadrati).

Inizializzazione della barrier

La barrier va creata nel `main` prima di lanciare i thread, specificando il numero di thread che devono raggiungerla prima che vengano sbloccati:

```

1  pthread_barrier_init(&barrier, NULL, T);
2  /* ... creazione e join dei thread ... */
3  pthread_barrier_destroy(&barrier);

```

Listing 4: Inizializzazione e distruzione della barrier

Fase map e sincronizzazione

Ogni thread eleva al quadrato gli elementi nel proprio segmento, poi chiama `pthread_barrier_wait()`. La barrier blocca ogni thread finché tutti e T non l'hanno raggiunta, garantendo che il reduce non inizi prima che il map sia completato da tutti. Lo standard POSIX garantisce che esattamente uno tra

i T thread riceva `PTHREAD_BARRIER_SERIAL_THREAD`: è l'unico meccanismo di selezione "uno tra N " offerto da pthreads senza mutex.

```
1 static void job(void *args) {
2     worker_args *w = (worker_args *)args;
3     pid_t tid = gettid();
4
5     printf("[Thread %d] IniziatO il map...\n", tid);
6     map(w->array, w->from, w->to);
7
8     int rc = pthread_barrier_wait(&barrier);
9     if (rc == PTHREAD_BARRIER_SERIAL_THREAD) {
10         printf("[Thread %d Seriale %d] IniziatO il reduce
11             ...\n", tid, rc);
12         w->result = reduce(w->array);
13     }
14     pthread_exit(NULL);
15 }
```

Listing 5: Funzione job: map, barrier e reduce

Nota: le slide riportano 332 833 500 000, che corrisponde a $N = 1\,000\,000$ elementi. Con $N = 800\,000$ (come specificato nell'esercizio) il valore corretto è però 266 266 800 000.

```
1 $ ./mapreduce
2 Somma dei quadrati (sequenziale): 266266800000
3 [Thread 58309] IniziatO il map...
4 [Thread 58310] IniziatO il map...
5 [Thread 58311] IniziatO il map...
6 [Thread 58312] IniziatO il map...
7 [Thread 58311 Seriale -1] IniziatO il reduce...
8 Somma dei quadrati (parallela): 266266800000
9 [OK] i risultati coincidono!
```

Listing 6: Test Esercizio 2